Vladmir capabilities and architecture

Technical review of branch main on October 14, 2026. This inventory describes what the code proposes and implements. It is not a certification, hardware-test result, or guarantee that the software works on every distribution.

Executive summary

Vladmir consists of a Linux Qt client, a Python backend using local Ollama, local voice/vision services, SQLite/private-file data, and Android/Wear clients. The design favors local inference and optional on-demand integrations. The repository includes limits and confirmations for several sensitive actions, but local integration still depends on configuration, service versions, and device state.

Area	Code status	Main dependency or limitation
Desktop chat	Implemented	Installed Ollama model; finite context
Chat, engineering, and teaching profiles	Implemented	Profiles do not create missing models or capabilities
Shared local memory	Implemented	Local persistence; limited memory/context
Knowledge library	Implemented	Partial extraction; quotas and supported formats
Voice and synthesis	Implemented	Whisper, Piper, FFmpeg, and system audio routing
Desktop/mobile calls	Turn-based implementation	Not real-time full-duplex audio
Vision and camera	Implemented	Frame capture; camera/model/FFmpeg must work
Web search/reading	Native integration	Internet and provider; pages may fail or change
GitHub	Explicit reading and editing	gh, network, account permission, content limits
Plugins	Native integrations	No generic extension marketplace
JetBrains MCP	Experimental local client	IDE server must be running and compatible
Agenda and reminders	Implemented locally	No external calendar sync
Home Assistant	Not implemented	No connector; read-only proposal is pending
Email	Draft preparation	Final sending occurs in the email client
Sentinel	Integrated local scan	ClamAV/signatures; limited to selected path
Software installation	Limited and confirmed request	Allowed package managers/sources; backup is an inventory
Code self-evolution	Isolated candidates	Review/promotion is not automatic
Adapter training	Protected experimental flow	Dataset, authorization, GPU, and evaluation required
Android	Kotlin/Compose source	SDK, build, server, TLS, pairing, and permissions
Companion/Wear	Native source and protocol	Pairing and background behavior need physical validation

1. Components

Desktop and core

src/mark/desktop.py implements the PyQt UI. desktop_ modules organize visio
n, memory, library,
plugins, JetBrains, Sentinel, evolution, and other dialogs. brain, core/, a
nd cognition/ handle
prompt routing, providers, tools, context, and cognitive functions. Calls s
tart from the same
desktop app.

The user can select General, Engineer, and Professor profiles. Specialized
conversations and
shared memory exist, but retention is limited by local databases, loaded co
ntext, and module
policy. The "instant/medium/high" selector chooses generation and context o
ptions; it does not
train the model or guarantee a measurable quality increase.

Local inference and efficiency

The default path uses HTTP to local Ollama and bundles no model weights. Mo
del profiles come
from models and ollama in config.json. keep_alive: 0 requests unloading aft
er a call to free
VRAM, which may increase latency on the next request. Performance depends o
n model,
quantization, CUDA/CPU backend, concurrency, and available RAM/VRAM.

Model names in the example are suggestions, not universal dependencies. A r
equest fails if the
corresponding Ollama tag is missing. The project does not automatically con
vert models into
"Heretic" variants, provide proprietary weights, or infer capabilities from
 a model name alone.

2. Chat, context, and specializations

- General: primary chat; accepts limited attachments, reasoning options, an
d tools available in
  the backend.
- Engineer: programming profile with workspace, missions, code reading, loc
al checks, and
  configured integrations.
- Professor: study and materials workflow with shared memory and optional l
ibrary references.
- Context: persisted conversations do not mean that the entire history is s
ent with every
  generation. Message count and model context are limited.
- Tools: the model does not automatically get unrestricted computer access.
 The application
  selects tools, permissions, and confirmations.

3. Library and extraction

cognition/library_extract.py limits an input file to 64 MiB, expanded archi
ve content to 32 MiB,
and extraction to 2 million characters. Chat has separate limits: 5 files,
6 MiB each, 8 MiB
total, and 400,000 extracted text characters.

Readers cover PDF, DOCX, EPUB, HTML, CSV, JSON, YAML, XML, many code format
s, and additional
formats through a universal parser. Extraction may depend on pdftotext, Tes
seract/OCR,
ffmpeg/ffprobe, and Whisper. Reading may be limited to pages, excerpts, or
metadata. Spreadsheet
formulas, images, archives, protected documents, and binaries are not neces
sarily interpreted as

a person would. Importing into the library does not update model weights.

4. Voice input/output

audio.py captures Pulse audio, mono at 16 kHz by default, calibrates a threshold, and detects
utterances. Whisper transcribes; tts.py synthesizes with Piper and may fall back to espeak-ng.
Device selection uses PulseAudio/PipeWire configuration. Missing models or devices cause an
error or fallback, not a perfect replacement.

Desktop/mobile calls send audio turns to the API. There is no guarantee of a continuously open
bidirectional stream, real-time interruption on every route, constant low latency, or perfect
coexistence with high-quality Bluetooth audio.

5. Vision

vision.py captures a frame using FFmpeg, checks that the bytes represent an image, and passes it
to a compatible vision profile. The source may be USB or RTSP(S). Live-stream/focus/quality
modules exist; frame cadence is limited. This is not continuous recording, recognition of every
object, persistent tracking, or a replacement for specialized sensors.

Typical dependencies: a device visible to Linux, camera permission, FFmpeg, a valid
authenticated URL, and a multimodal model. A valid URL or enumerated camera does not prove the
stream is reachable.

6. Web, integrations, and plugins

The web plugin searches for and reads public pages. The code filters non-public destinations to
reduce SSRF; as a result it is not an intranet browser or unrestricted network access.

The github plugin reads projects available through the API/gh; editing is an explicit action and
requires confirmation to write. The account needs suitable authentication and scope. JetBrains
MCP accepts an HTTP server on localhost, lists tools, and asks for confirmation before running
them. Available operations depend on what the IDE exposes.

The plugin registry (plugins.py) is a small list of native capabilities. The UI says that
arbitrary external plugins are not executed. There is no universal sandbox for marketplace
extensions.

7. Auxiliary features

- Agenda: local SQLite for tasks, dates/times, and completion; reminders are triggered by the
  application and do not sync with an external calendar.
- Email: EmailComposerDialog builds a mailto: draft; the user reviews and sends it in the
  default mail client.
- God’s Eye View: on-demand local geospatial tool; data and visualization depend on selected
  providers and network access.

- Sentinel: uses ClamAV on the selected path and stores a local report. It is not an EDR suite
  or a whole-system validation. VirusTotal receives no hash by default.
- Packages: package_install.py restricts managers/sources and creates an inventory before
  authorized installation. The inventory cannot undo every package side effect.

8. Mobile API and authentication

mobile_api.py provides health, model, agenda, history, pairing-status, session, chat-completion,
voice, and Wear-telemetry endpoints. The API is local by default. Authentication binds sessions
to a device; critical actions are blocked for non-owner clients. Request bodies, text, and
attachments have explicit limits. Concurrent requests may receive HTTP 429.

Remote access requires deliberate TLS/tunnel, address, and authorization configuration. No
personal endpoint is embedded. Physical connectivity must be verified with the server app
running, phone available, valid certificates/address, pairing, and appropriate network rules.

9. Android, Companion, and Wear OS

The Android client includes chat, photo capture, voice, attachments, and optional ARCore. A
device without ARCore support or Google Play Services for AR cannot run AR mode. The current AR
map lasts only while the screen remains active; it is not persisted across sessions.

Companion can relay selected notifications, including a limited WhatsApp preview, when the user
grants Android Notification Listener Access. Authenticated Bluetooth depends on pairing and the
Android service. The UI diagnostic says automatic reconnection is not yet enabled; manufacturer
battery management may suspend the process. Wear enumerates available sensors, but availability
and permissions vary by model. Heart-rate or orientation readings are not medical measurements.

Source code does not prove that APKs have been built, installed, approved, or that a connection
remains stable with the screen off. That requires builds/signing and tests on specific devices
and versions.

10. Privacy and security

Data is stored under local user directories. The project recommends config mode 0600, loopback
API binding, and an authenticated tunnel for remote access. Web, GitHub, Honcho, and VirusTotal
send data to third parties when used; Honcho and VirusTotal are disabled by default in the
example config. Review each service’s policy before enabling it.

Application safeguards do not replace OS updates, a full antivirus, backups, permission
management, or code review. Report vulnerabilities privately according to SECURITY.en.md.

11. Self-evolution and maturity

evolution/engine.py runs an isolated experiment, collects comparisons, and stores reports.
Candidate decisions remain REVIEW_REQUIRED; promotion is not authorized. Applying changes to the
runtime is a separate maintenance action. The panel has start/stop, and the systemd routine is
opt-in; do not assume it is active in a new installation.

automatic_training.py requires an explicit authorization contract, owner-approved dataset, file
hashes, a holdout set, and bounded parameters. Energy/quality/VRAM measurements may be
unavailable; without valid measurements, a candidate must not be considered efficient. The
process does not update active weights on its own.

12. Confirm capabilities in an installation

1. Record the installed commit and operating system.
2. Run --check and keep output free of secrets.
3. Confirm local Ollama and models.
4. Test each feature separately with its service/device available.
5. For the API, validate /health, /v1/models, and authentication without exposing a token.
6. For camera/audio/Bluetooth, perform a physical test and inspect device state.
7. Record the result, error, latency, dependencies, and test scope.

13. Repository structure, tests, and integration automation

The repository has source (src/mark/), tests (tests/), operations scripts (scripts/), and
separate Android/Wear projects. It has 37 tests/test_.py files using unittest, plus unit and
integration smoke-test scripts. The existence of an automated suite does not make the file count
a coverage metric.

At commit d9b3358, there were no workflows under .github/workflows/ and no published coverage
configuration/report. The accurate status is "tests exist; CI and coverage percentage are not
published," not "no tests" or "zero coverage." Integration smoke tests depend on Ollama, models,
and local resources; unit tests do not prove the full integration or hardware works.

The root README includes the local test command. This document does not claim tests were run
during inspection.

14. Source references

src/mark/desktop.py, src/mark/brain.py, src/mark/core/, src/mark/cognition/,
src/mark/mobile_api.py, src/mark/attachment_limits.py, src/mark/plugins.py, src/mark/github.py,
src/mark/jetbrains_mcp.py, src/mark/package_install.py, src/mark/sentinel.py,
src/mark/vision.py, src/mark/audio.py, src/mark/tts.py, src/mark/agenda.py, src/mark/evolution/,
mobile-android/, vladmir-companion/, vladmir-wear/.